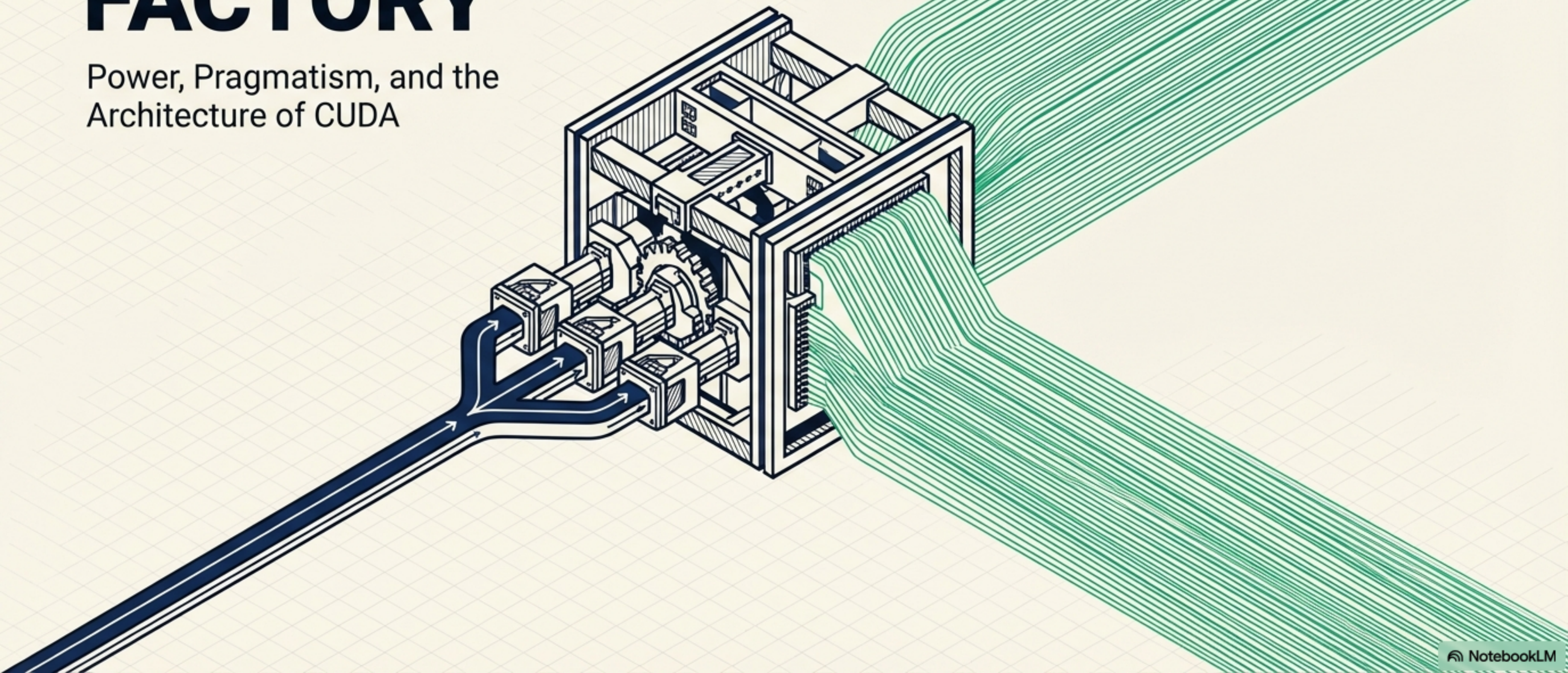
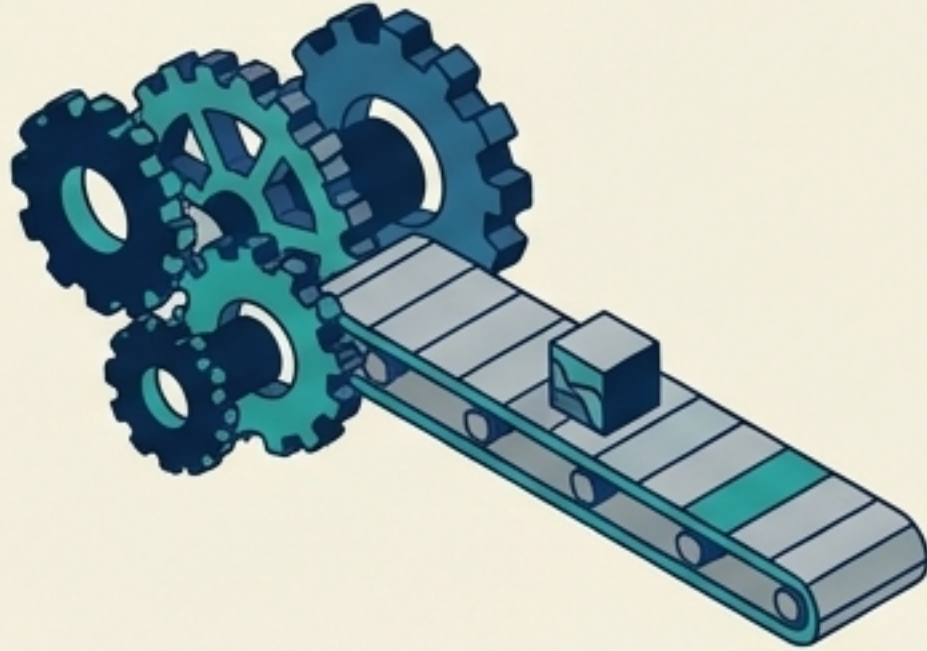


THE SILICON FACTORY

Power, Pragmatism, and the
Architecture of CUDA



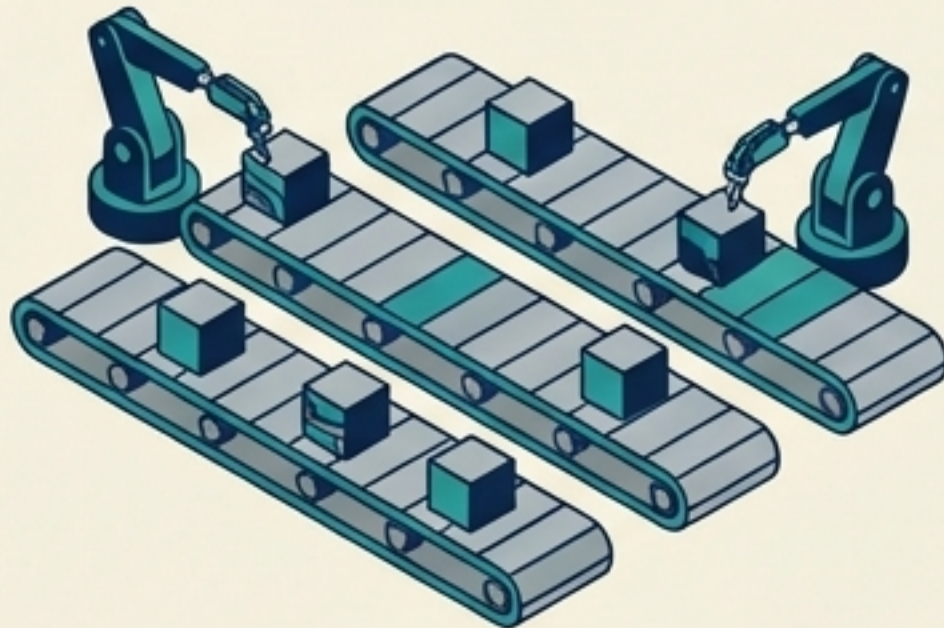
Breaking the sequential speed limit



Serial Computation (CPU):

A single CPU executes one instruction at a time. Physical limits on power and heat prevent the single lane from getting continuously faster.

Difference: Linear, one-step-at-a-time process. Limited by single-core speed.

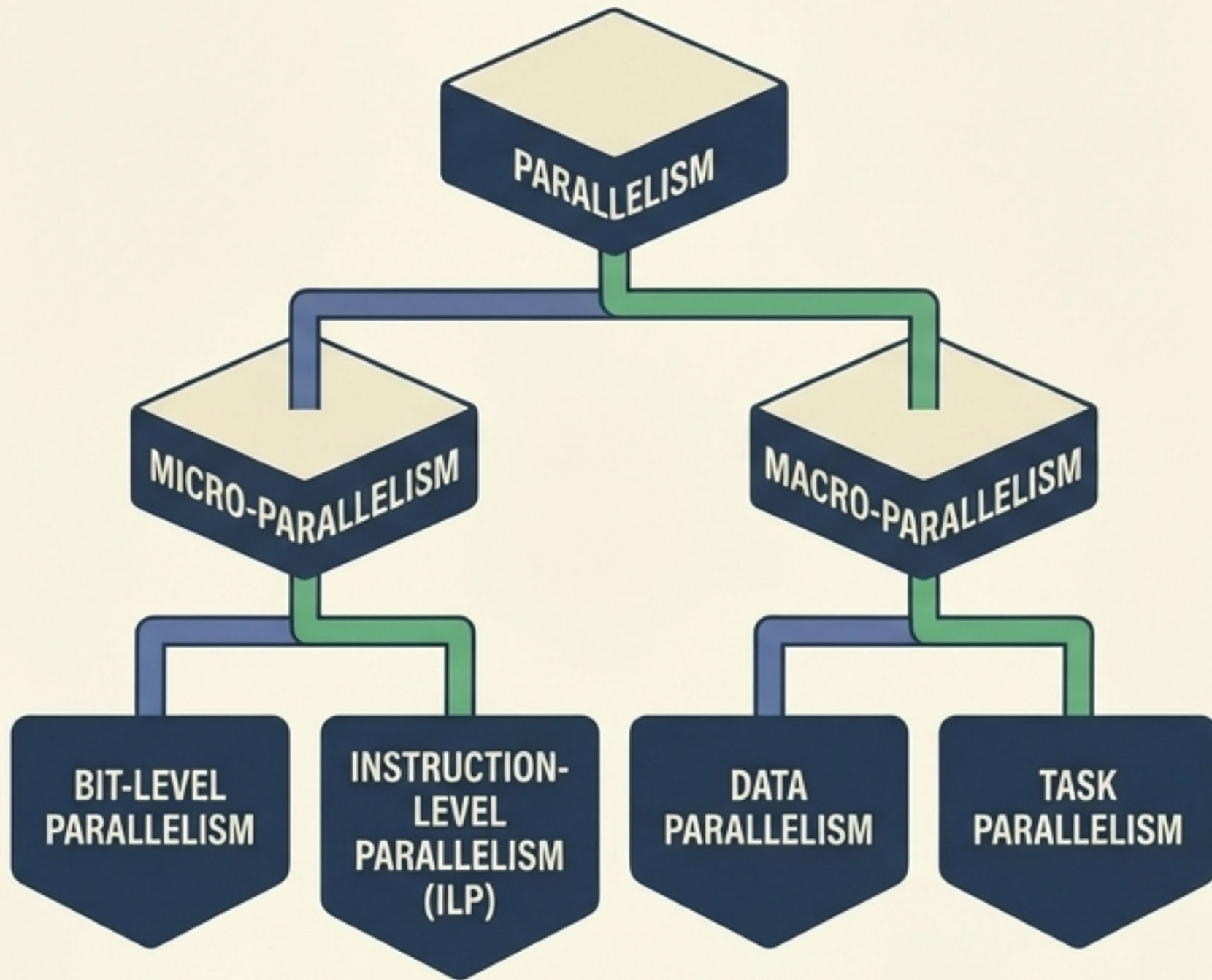


Parallel Computation:

Breaks large problems into independent parts processed simultaneously across multiple computing elements.

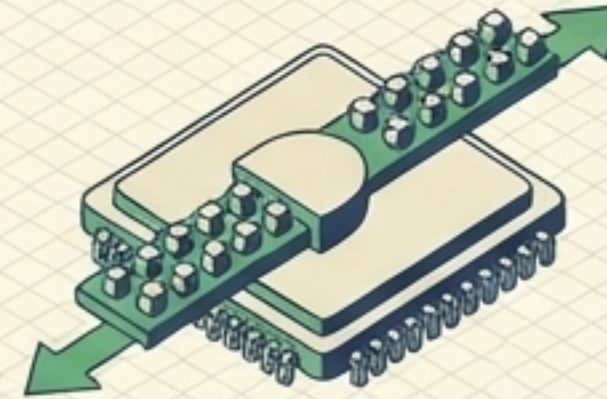
Difference: Multi-step process. Limited by problem divisibility and overhead.

Types of Parallelism: A Flowchart



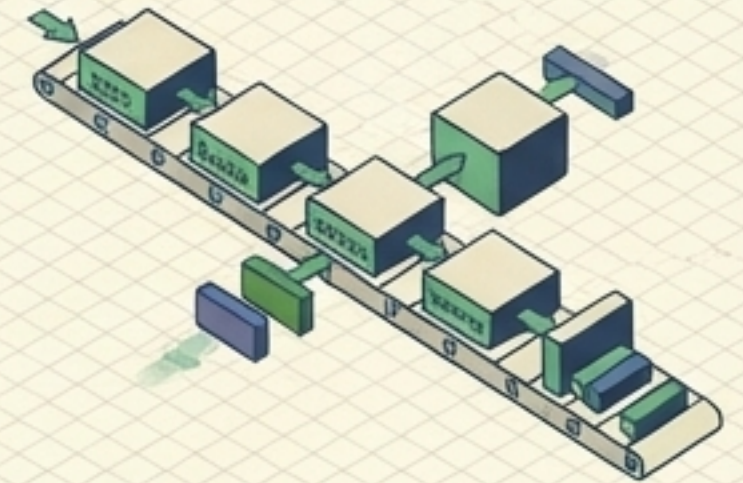
Micro-Parallelism: Fundamental Operations

Bit-Level Parallelism



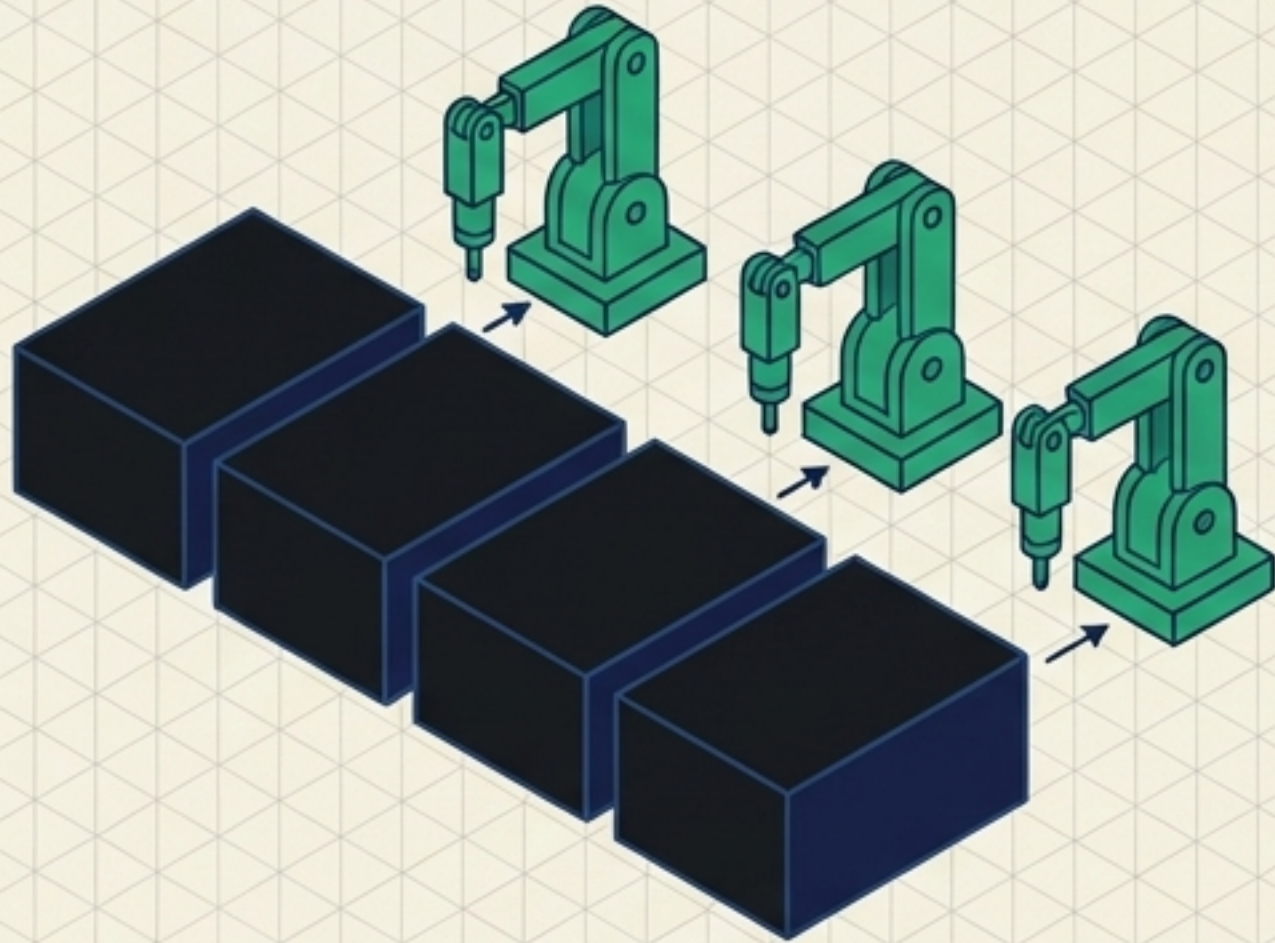
Processing multiple bits of a data word simultaneously (e.g., 64-bit vs. 32-bit arithmetic). Increases the amount of data handled per cycle.

Instruction-Level Parallelism (ILP)



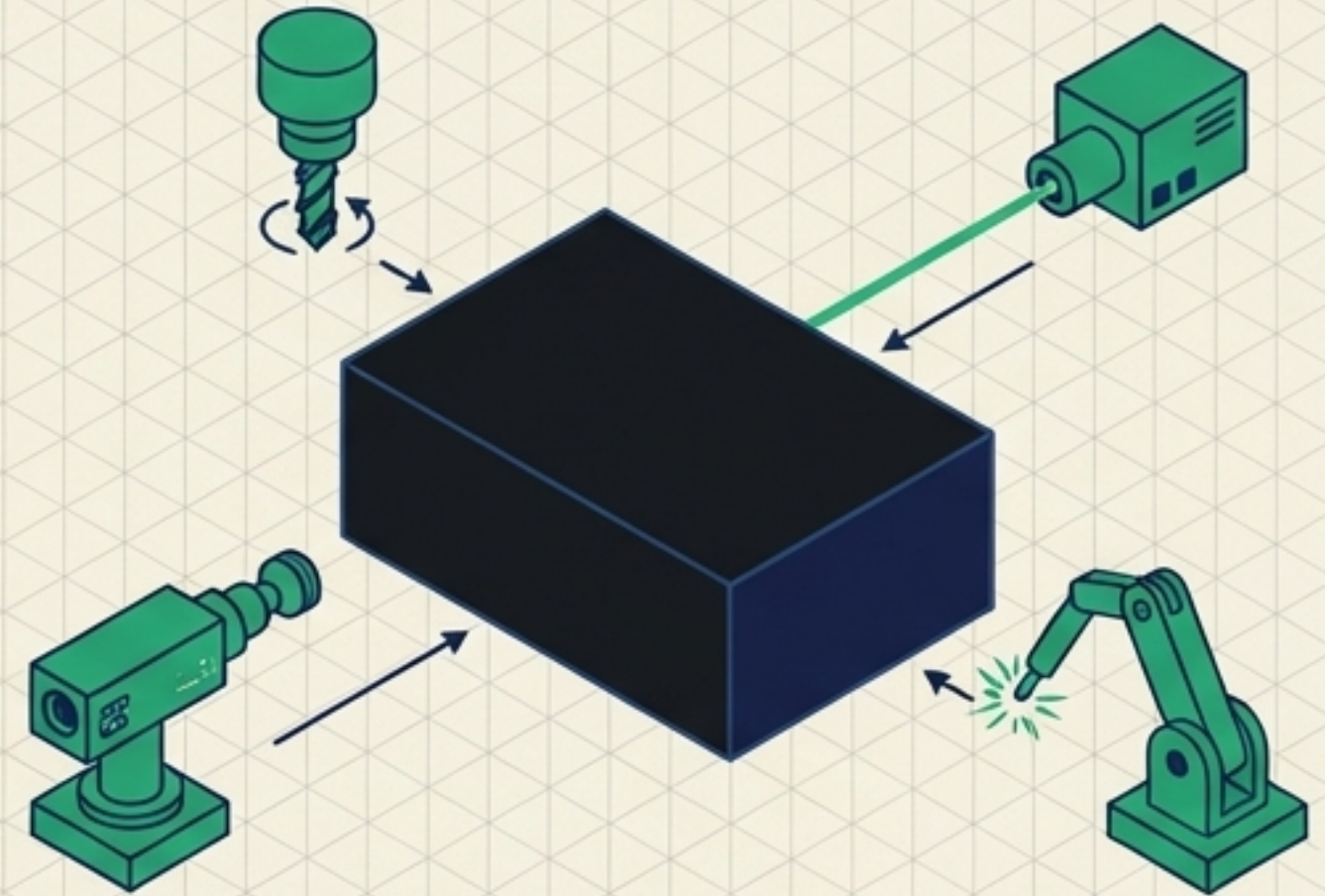
Executing multiple instructions from a single stream simultaneously using techniques like pipelining, superscalar execution, and out-of-order execution.

Macro-parallelism divides the workload



Data Parallelism

Dividing an array of 'n' elements into equal parts assigned to different processors.



Task Parallelism

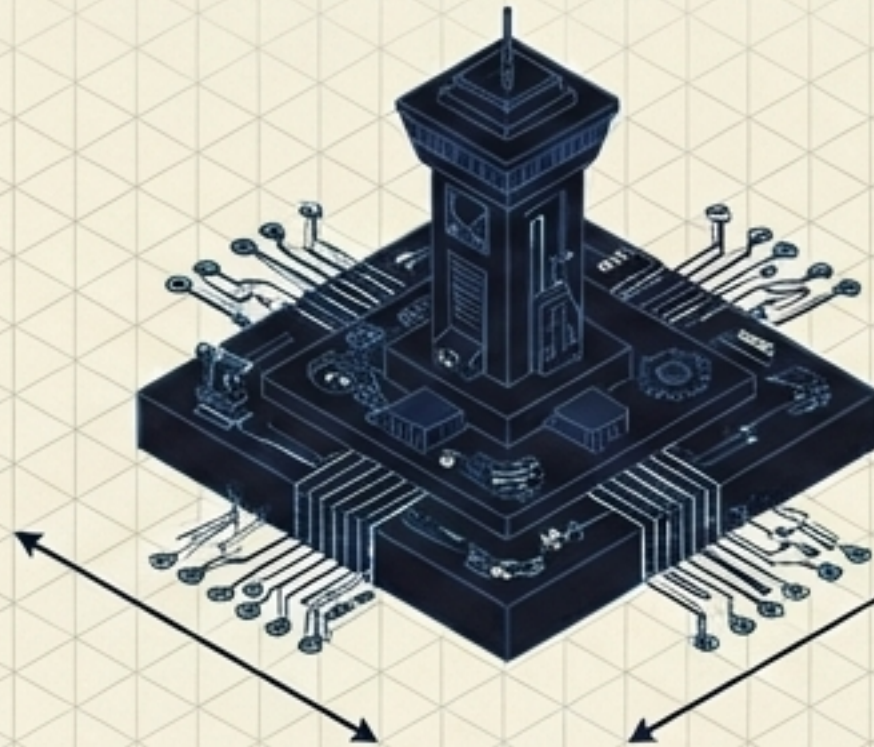
Multiple processors executing completely different threads or processes on the same data pool.

The Pragmatic Formula: Splitting a task of time T across N processors takes $T/N + \text{Merge Overhead Time}$. Merging results creates a hard ceiling on speed.

CPU v/s GPU

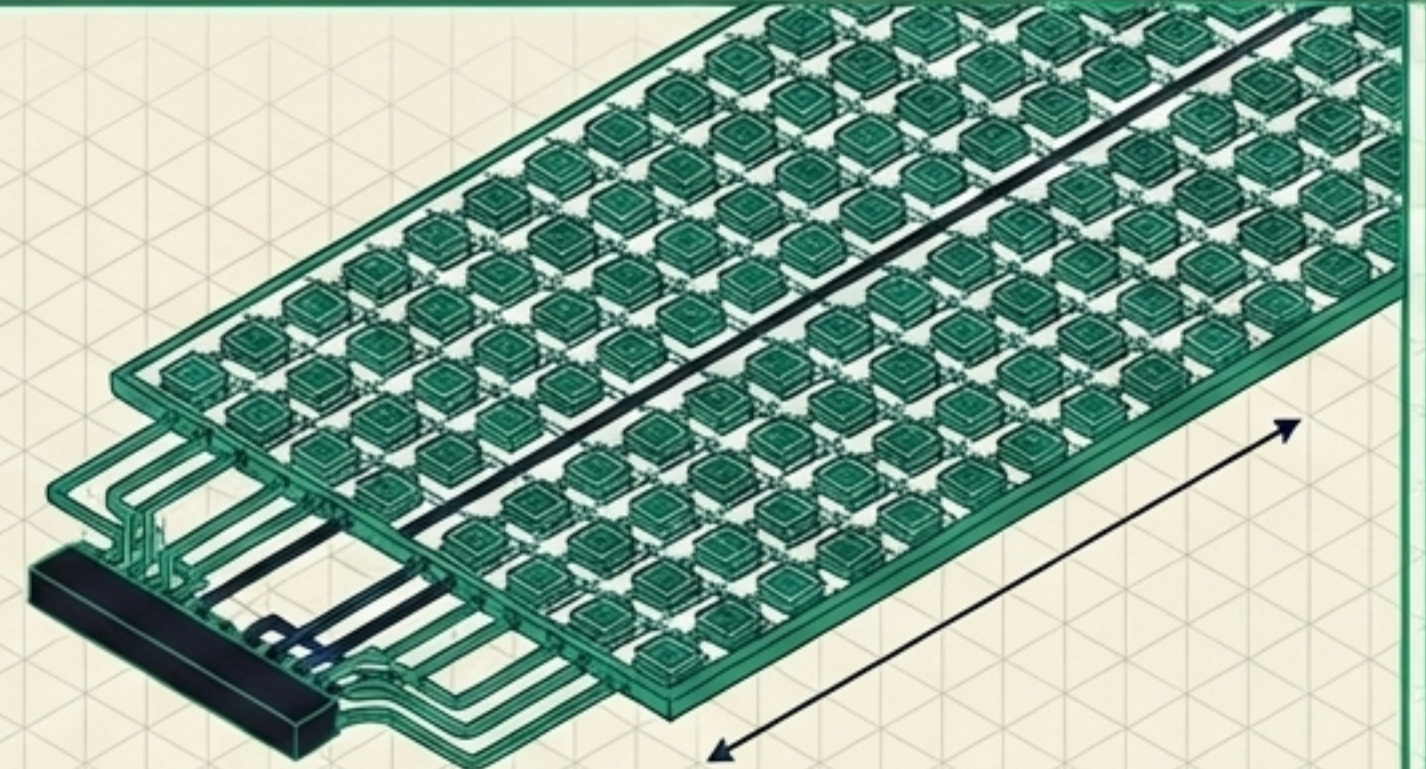
The hardware divide dictates the computing strategy

CPU (Central Processing Unit)



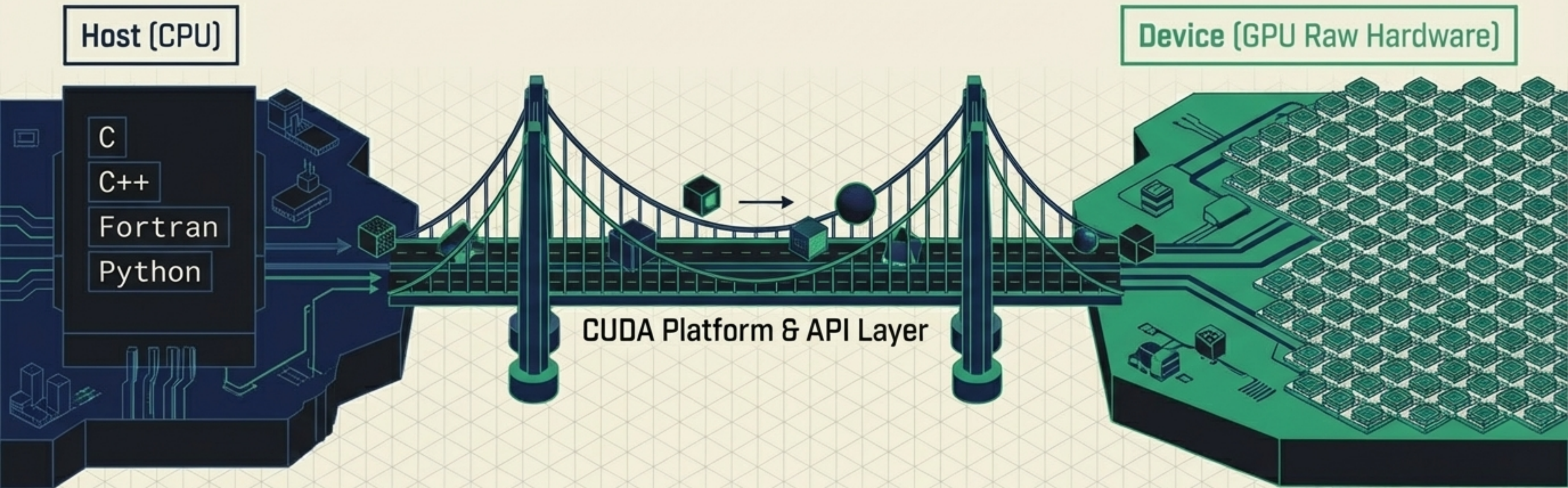
Core Count:	→ 1 to 16 highly powerful cores.
Transistor Allocation:	→ Dedicated to massive data caching and complex flow control.
Performance Profile:	→ Dominates general-purpose tasks, serial calculations, and complex branching logic. The Executive Chef.

GPU (Graphics Processing Unit)



Core Count:	→ 100s to 1,000s of simpler cores.
Transistor Allocation:	→ Devoted almost entirely to raw Data Processing Units.
Performance Profile:	→ Dominates massive-scale, repetitive calculations and highly parallel chunks like image processing.

CUDA transforms graphics cards into supercomputers



What it is

Compute Unified Device Architecture.
A proprietary parallel computing platform and API by NVIDIA.

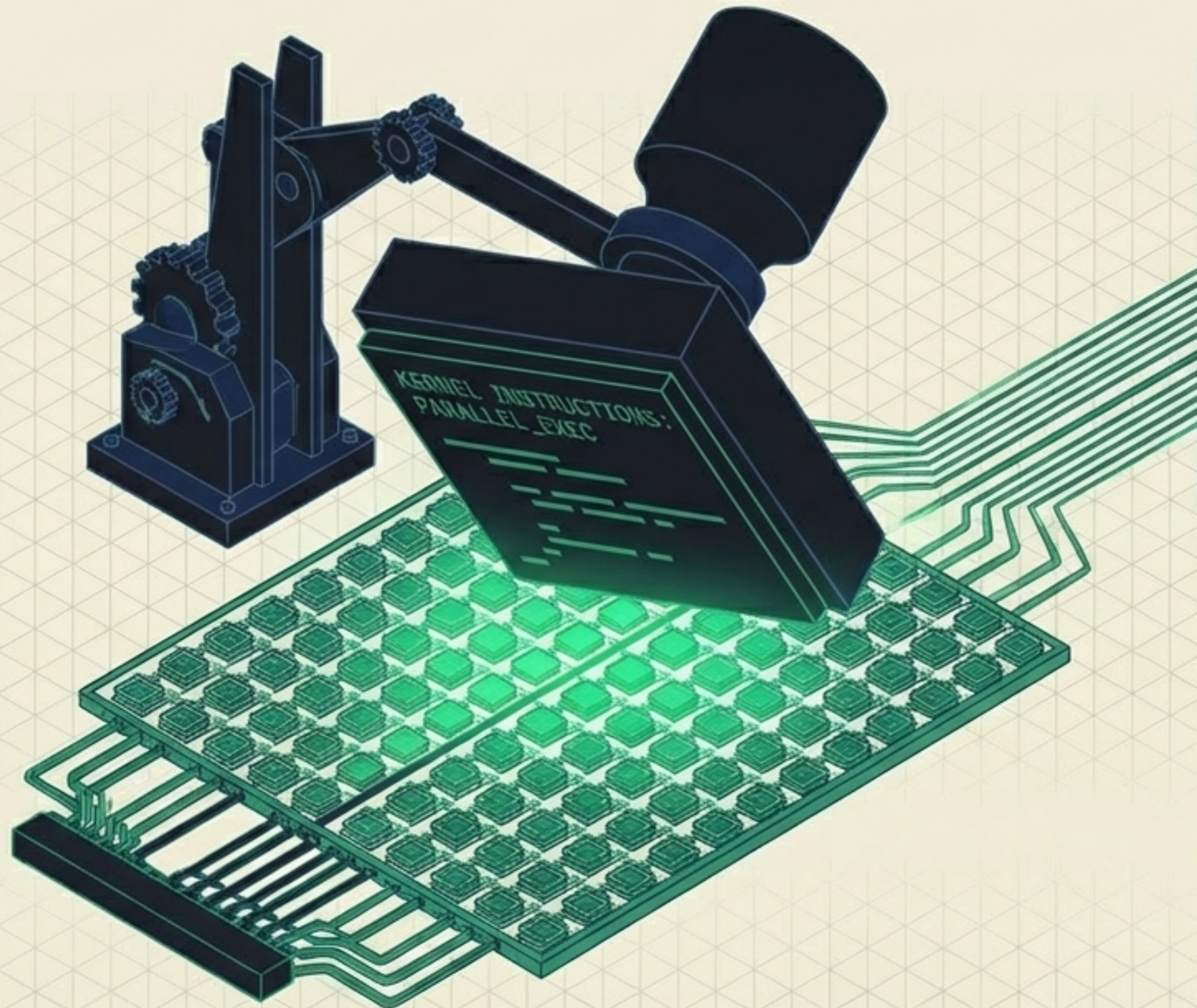
The Purpose

Allows developers to bypass graphics rendering engines and use GPUs for accelerated general-purpose scientific math.

The Ecosystem

Not just a language. It is a software layer managing host/device access, including compilers (nvcc), libraries, and developer tools.

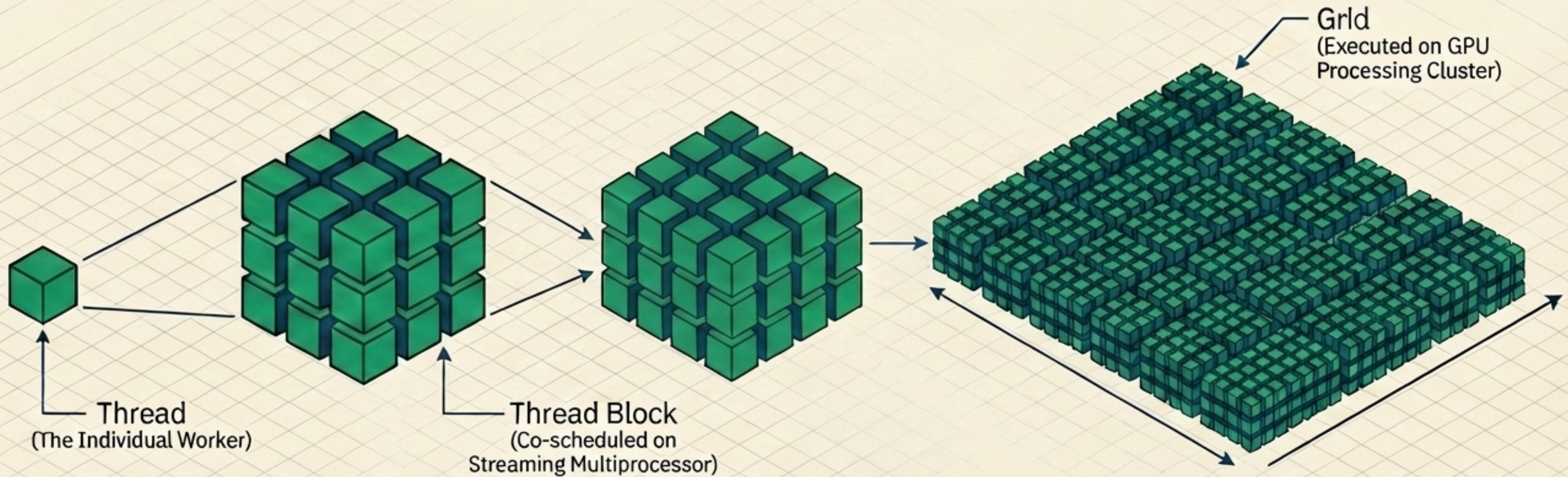
Compute Kernels are the blueprints of parallelization



```
__global__ void myKernel() {  
    // parallel computation  
    instructions  
}
```

- ➔ **The Concept:** A kernel is a function written once, but executed N times in parallel by the GPU.
- ➔ **The Signal:** The `__global__` specifier flags the compiler that this function belongs on the Device (GPU), not the Host (CPU).

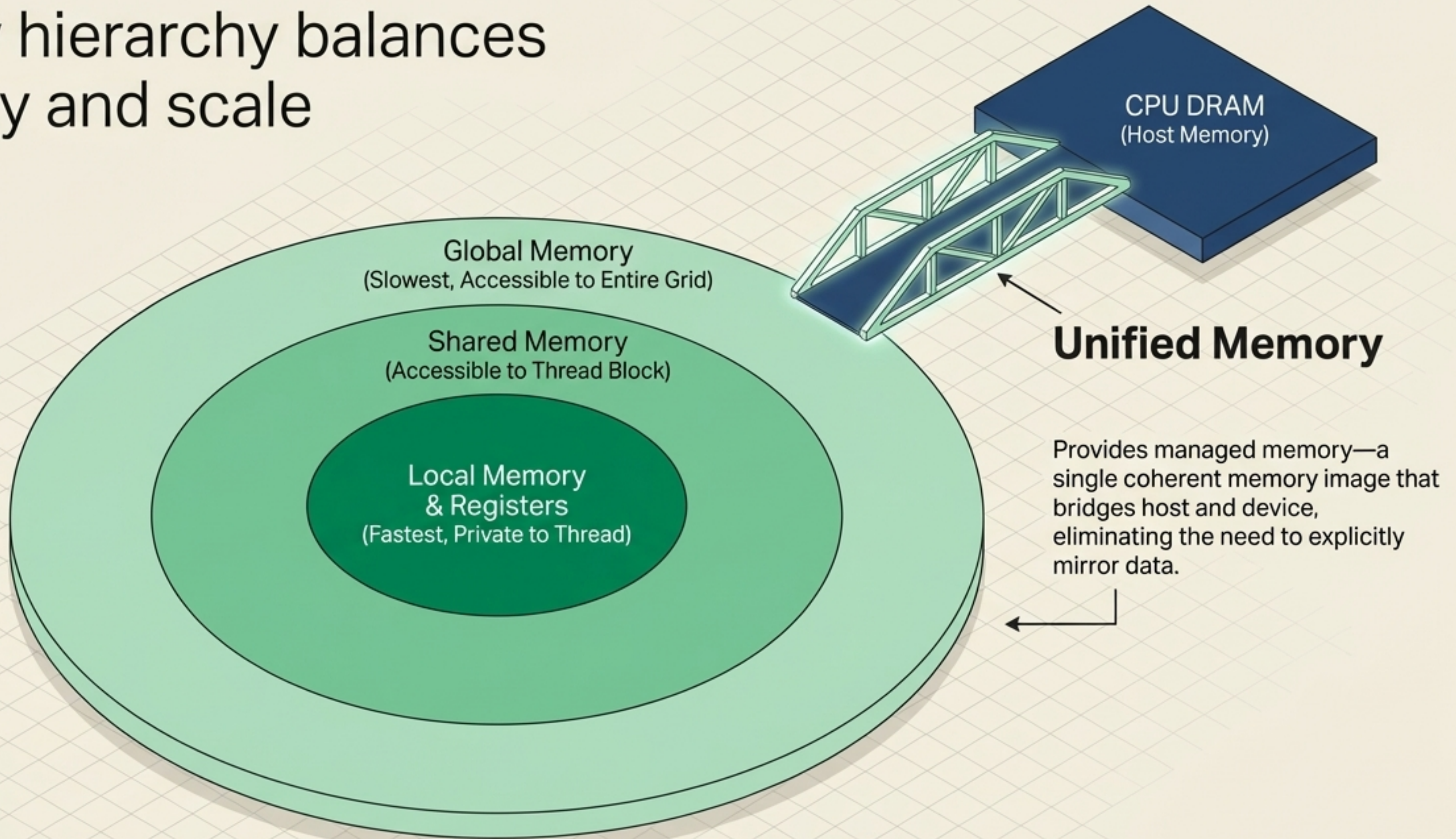
Thread hierarchy organizes the computational army



Execution Syntax: `<<< blocksPerGrid, threadsPerBlock >>>`

By organizing threads into blocks, and blocks into grids, developers can optimize data locality and resource sharing across the GPU.

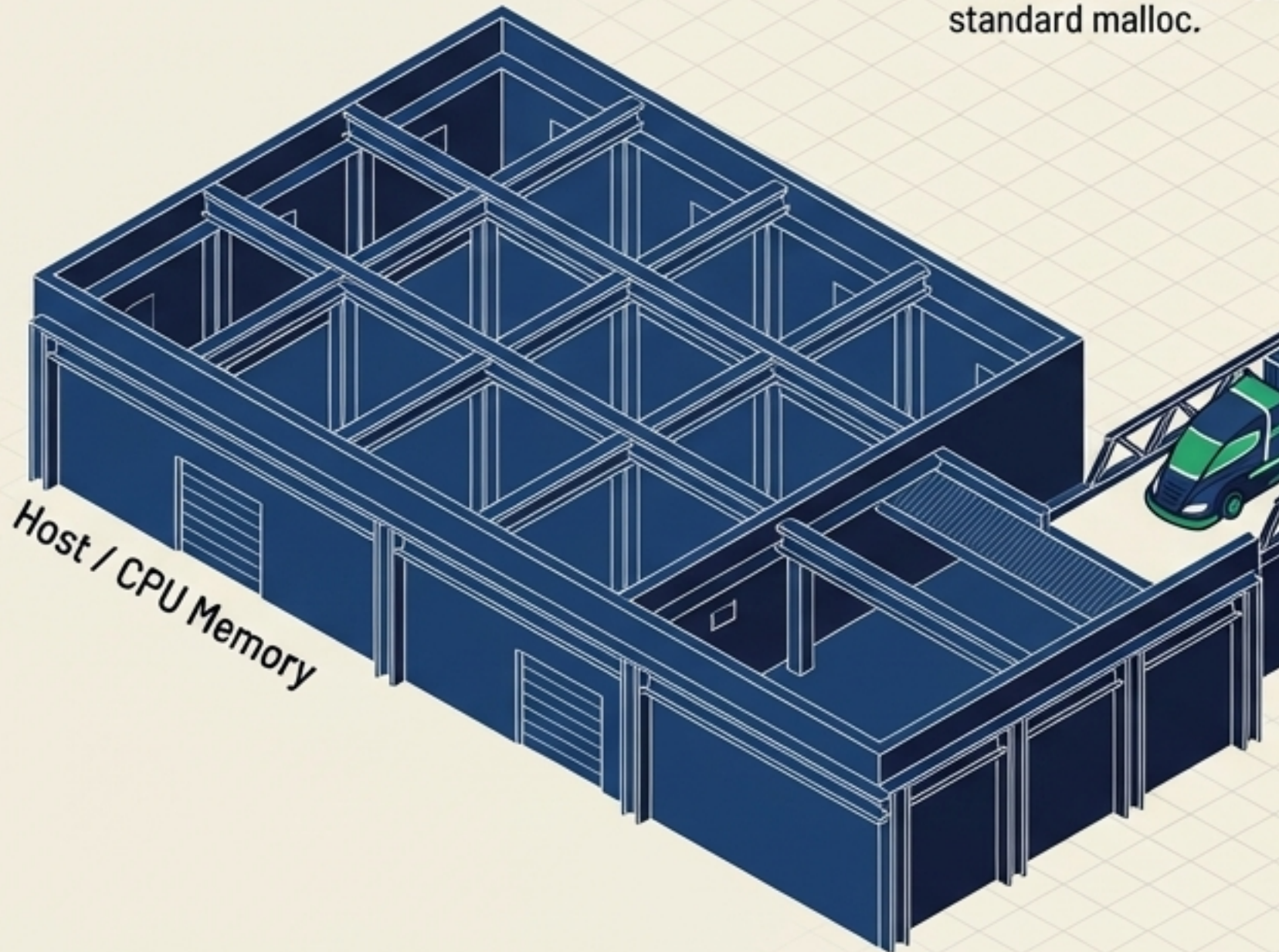
Memory hierarchy balances proximity and scale



Controlling the factory logistics: Memory functions

1. `cudaMalloc(&p, size)`

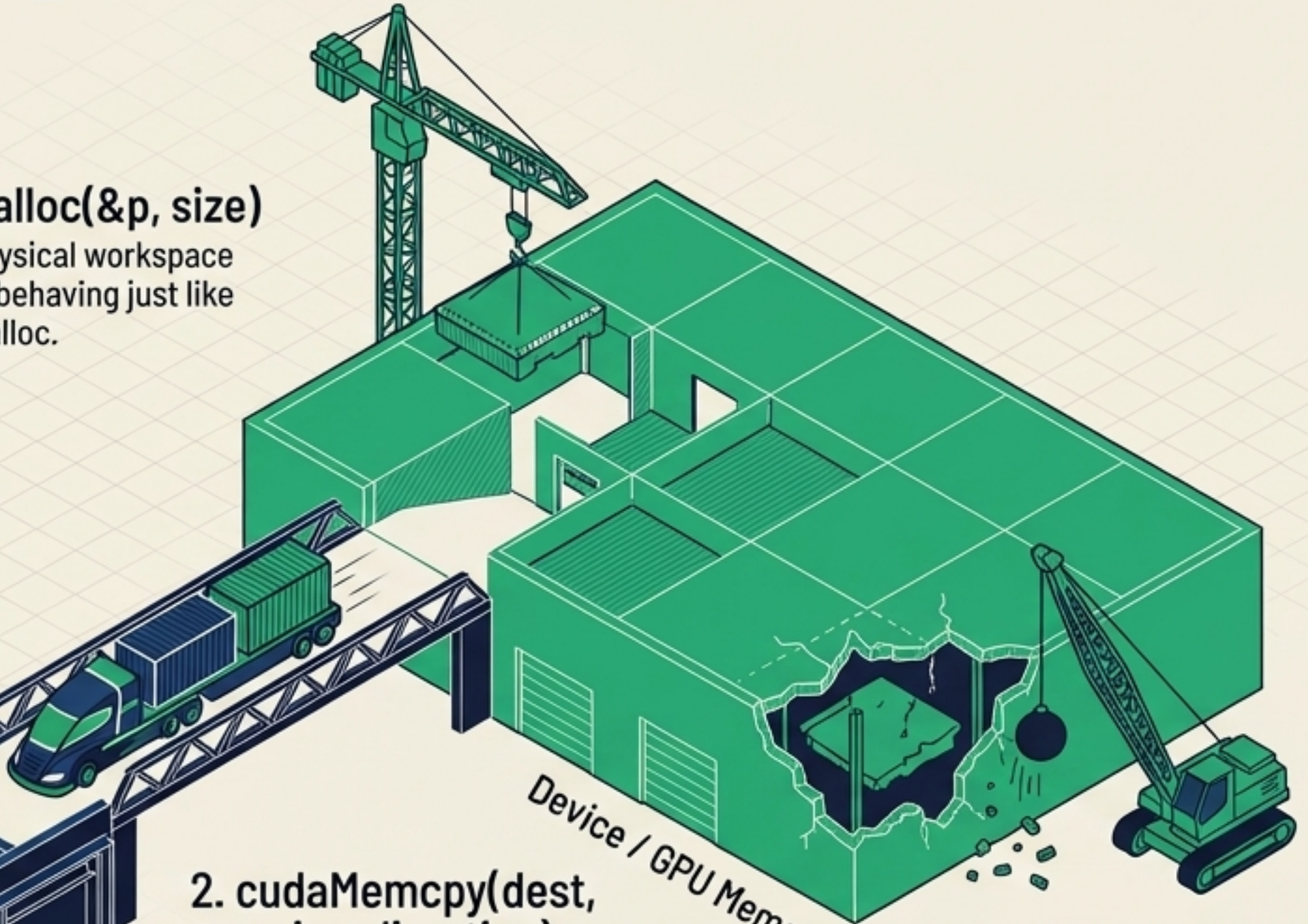
Allocates physical workspace on the GPU, behaving just like standard malloc.



Host / CPU Memory

2. `cudaMemcpy(dest, src, size, direction)`

Trucks data between Host and Device spaces.

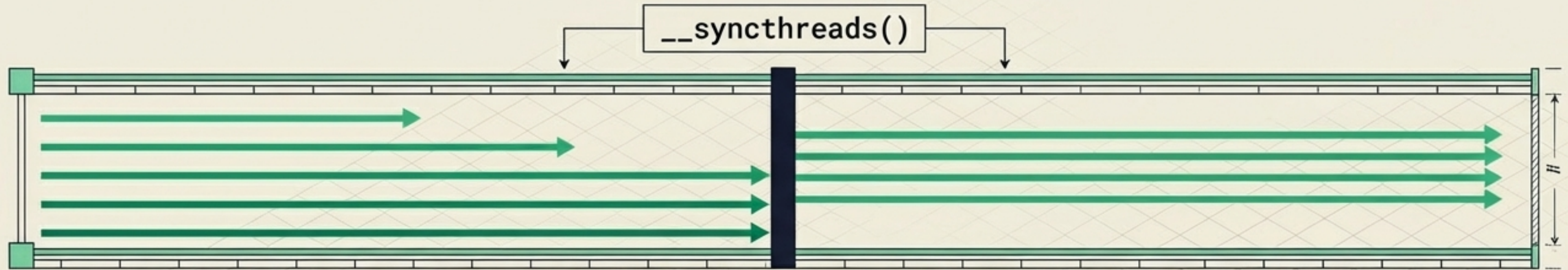


Device / GPU Memory

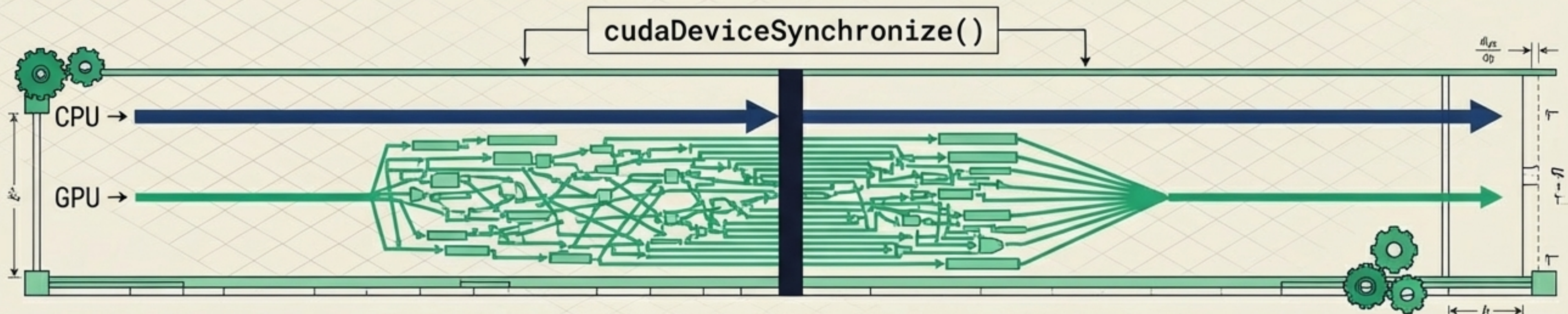
3. `cudaFree(p)`

Demolishes and deallocates the GPU memory when the compute is completely finished.

Controlling the rhythm: Synchronization functions



Synchronizes thread operations within a block. Pauses fast threads until the slowest thread finishes its preliminary calculations. Puts everyone on the same page.



Synchronizes Host and Device. Forces the asynchronous CPU to halt and wait for the GPU to finish its workload before continuing the serial program.

Findings of the Tasks!

Task: Printing 'Hello World' 1024 times

- CPU Execution Time: 0.00000 seconds
- GPU Execution Time (<<<4,256>>>): 0.11300 seconds

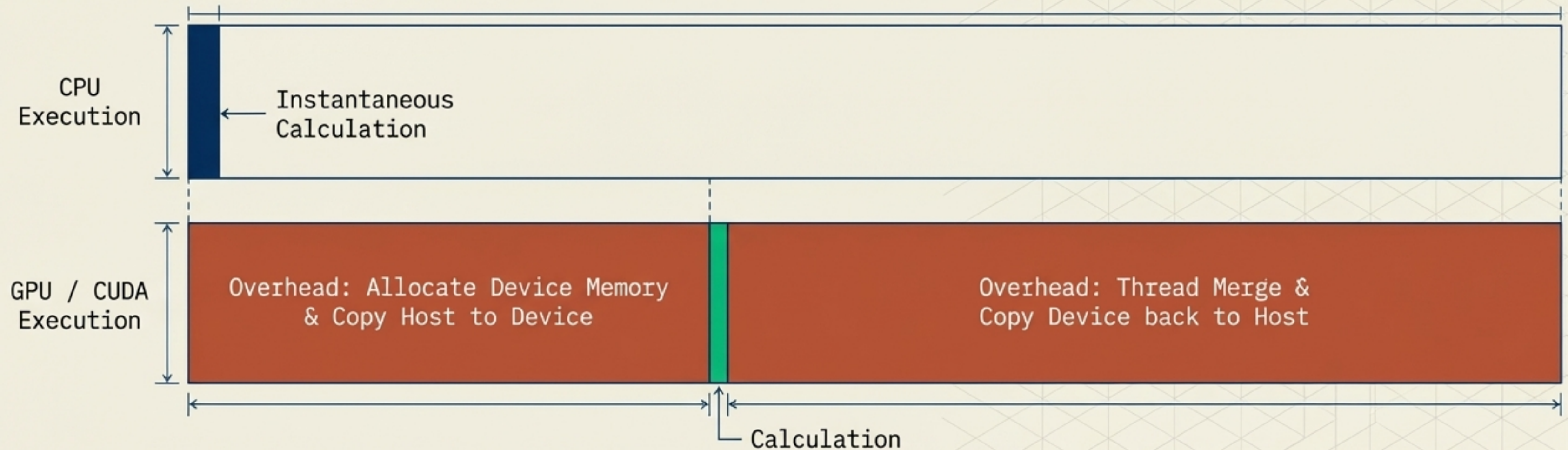
Expectation vs Reality

- Expectation: GPU should be significantly faster than CPU for parallel tasks.
- Reality: GPU execution was slower due to overhead (memory transfer, kernel launch) for these specific, small workloads.

Task: Matrix Addition (32x32)

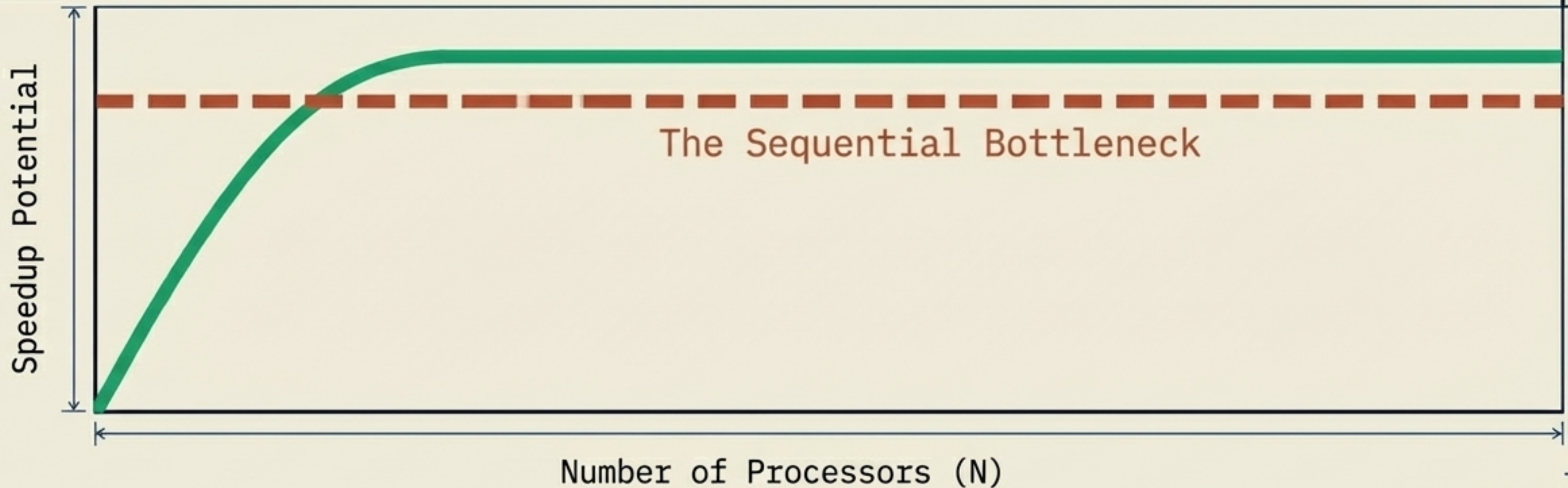
- CPU Execution Time: 0.00000 seconds
- GPU Execution Time: 0.007 seconds

Unmasking the merge tax and logistical overhead



- **The Hidden Cost:** Inter-process communication and synchronization incur overheads often orders of magnitude higher than the actual math.
- **The Logistical Trap:** The GPU did the math instantly, but the logistics of moving data to the factory and shipping it back consumed 90% of the execution time.

Amdahl's Law dictates the limits of parallelization



Amdahl's Law: Increasing the number of processors yields diminishing returns. No matter how many thousands of cores your GPU has, the total task speed is permanently capped by the strictly serial portions of the code (like moving memory and merging thread results).

Thank You for Reading

CUDA Task
Jashesh Pandya
25ME10097